

MASTERING PROMETHEUS

Prometheus가 관측값을 믿을 만한 신호로 만드는 과정

데이터가 태어나 저장되고, 대상을 찾고, 경고가 사람에게 도착하기까지를 한 흐름으로 봅니다.

오늘 따라갈 한 줄

대상을 찾는다 → 값을 긁는다 → 시계열로 저장한다 → 규칙을 평가한다 → 알림을 묶어 사람에게 보낸다.

먼저 짚을 용어 넷

- **메트릭**: 측정 대상의 이름
- **라벨**: 같은 메트릭을 쪼개는 꼬리표
- **스크레이프**: 서버가 `/metrics` 를 주기적으로 가져가는 일
- **알림**: 조건이 일정 시간 참일 때 생기는 상태

오늘의 순서

수집 무엇을 긁을지 고르고, 값은 누가 만드는가
저장 그 값이 어떤 모양으로 쌓이고 질의되는가
알림 두 프로그램의 책임 분리

맨 위 문장의 다섯 단계를 순서 그대로 따라갑니다. 값이 태어나는 자리에서 시작해, 사람에게 도착하는 데서 끝납니다.

STEP 1 / 3

수집 — 무엇을 긁고, 값은 누가 만드는가

대상을 찾는다 → 값을 긁는다

서비스 디스커버리

relabeling

Exporter · collector

디버깅

서비스 디스커버리는 후보만 만듭니다

Prometheus는 스스로 대상을 알지 못합니다. "어디를 굽어야 하는지" 주소 목록을 대신 받아 오는 일이 서비스 디스커버리입니다.

파드는 뜨고 지고 IP도 계속 바뀝니다. 그래서 주소를 설정 파일에 적어 두는 대신, 쿠버네티스나 클라우드의 API에 주기적으로 다시 물어봅니다.

API에 묻는다 → 후보 목록 → relabeling이 고른다 → 남은 주소를 굽는다

발견

후보 목록과 `__meta_*` 라벨을 받습니다.

아직 아무것도 굽지 않은 상태입니다.

선별

relabeling이 남길 대상을 결정합니다.

여기서 후보 수가 줄어듭니다.

스크레이프

살아남은 최종 주소로 요청합니다.

이때 비로소 값이 생깁니다.

SD는 후보만 냅니다. 값은 exporter가 만들고, 무엇을 굽을지는 relabeling이 정합니다 — 셋의 책임이 다릅니다.

relabeling은 두 번 돌고, 거르는 것이 다릅니다

이름이 닮아 헷갈리지만 **놓인** 자리가 다르고, 그래서 거르는 대상도 다릅니다.

앞의 것은 **굼기 전에** 돌아 대상을 고르고, 뒤의 것은 **저장하기 전에** 돌아 시계열을 고릅니다.

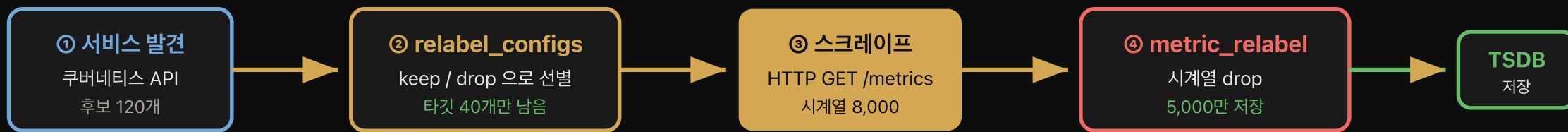
	relabel_configs	metric_relabel_configs
언제 도는가	스크레이프 전	저장 전
무엇을 거르는가	타겟 — 굼을 대상	시계열 — 저장할 값
무엇이 줄어드는가	스크레이프 부하 · 네트워크	카디널리티 · 디스크
못 하는 일	이미 받은 값은 못 거름	굼는 것 자체는 못 막음

특정 파드를 *아예* 굼지 않으려면 어느 쪽일까요?

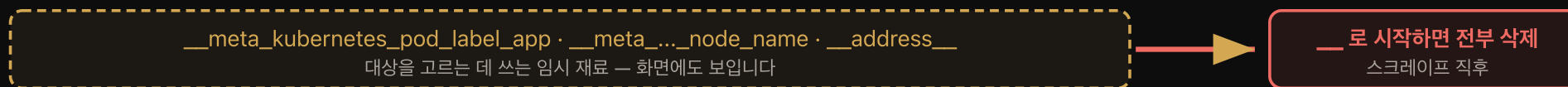
정답: **relabel_configs** 입니다. metric 쪽은 이미 굼은 뒤라 부하가 그대로 듭니다.

후보에서 저장까지, 무엇이 걸러지는가

단계마다 걸러지는 것도, 남는 라벨도 달라집니다.



라벨이 살아 있는 구간



남기려면 옮겨 담아야 합니다

source_labels: [__meta_kubernetes_pod_node_name] → target_label: node (이제 저장됩니다)

주소를 받는다

값을 대상을 고른다

값을 가져온다

저장할 값을 고른다

여기까지는 타깃 개수가 줄어듭니다

여기서는 시계열 개수가 줄어듭니다

규칙 하나는 부품 다섯으로 읽습니다

규칙은 "어떤 라벨을 보고, 무엇에 맞으면, 어떻게 한다"의 반복입니다.

아래는 **app=api** 라벨이 붙은 파드만 굵겠다는 규칙입니다.

```
relabel_configs:
- source_labels:
  [__meta..._label_app]
  regex: api
  action: keep
```

이 네 줄은 이렇게 읽습니다.

1. 파드에 붙은 **app** 라벨 값을 본다
2. 그 값이 **api** 와 정확히 같은가
3. 같으면 남기고, 아니면 버린다

필드	뜻
<code>source_labels</code>	어떤 라벨의 값을 볼 것인가
<code>regex</code>	그 값이 이 패턴에 맞는가 (기본 <code>(.*)</code>)
<code>action</code>	맞으면 무엇을 할 것인가 (기본 <code>replace</code>)
<code>target_label</code>	어느 라벨에 쓸 것인가
<code>replacement</code>	실제로 쓸 값 (기본 <code>\$1</code> 첫 캡처)

`regex` 는 양쪽 끝이 앵커링됩니다. 부분 일치를 원하면 `.*api.*` 로 풀어야 합니다.

후보 수를 바꾸는 액션은 둘뿐입니다

`action` 이 대상을 줄일지, 라벨만 손댈지를 가릅니다.

현행 문서는 액션을 11개 정의하지만 실무에서 손이 가는 것은 아래 다섯입니다.

```
action: keep      # 남길 것만
action: drop      # 뺄 것만

action: replace # 값 옮기기
action: labeldrop
action: labelmap
```

`keep` 과 `drop` 은 서로 반대라 하나로 같은 일을 표현할 수 있습니다. 정규식이 단순해지는 쪽을 고르는 것이 관례입니다.

액션	하는 일	후보 수
<code>keep</code>	매칭되는 것만 남김	줄어들
<code>drop</code>	매칭되는 것을 버림	줄어들
<code>replace</code>	읽은 값을 옮겨 씀	안 바뀜
<code>labeldrop</code>	라벨 이름을 지움	안 바뀜
<code>labelmap</code>	라벨 이름을 복사	안 바뀜

`replace` 는 값을, `labeldrop` 은 라벨 자체를 다룹니다. 이름이 비슷해 자주 헷갈리는 자리입니다.

쿠버네티스에서는 Operator가 대신 씁니다

지금까지 본 SD 설정과 relabeling 규칙을 손으로 쓰지 않습니다.

사람은 **ServiceMonitor**라는 쿠버네티스 리소스에 "어느 서비스의 어느 포트를 긁어라"만 적고, Operator가 그것을 앞 장에서 본 **실제 scrape 설정**으로 번역합니다.

ServiceMonitor 작성 → Operator가 번역 → **prometheus.yaml** 생성 → 사이드카가 재적재

긁을 대상	사람이 쓰는 것	Operator가 하는 일
K8s Service·Pod	<code>ServiceMonitor</code> · <code>PodMonitor</code>	selector를 <code>kubernetes_sd</code> + relabel 규칙으로 번역
블랙박스 프로빙	<code>Probe</code>	대응 SD와 스크레이프 설정 생성
EC2 · 사내 SD	<code>additionalScrapeConfigs</code>	번역 없음 — 처음부터 SD 문법으로 직접

두 경로는 번역 단계가 있느냐로 갈립니다. `additionalScrapeConfigs` 는 YAML을 직접 받지 않고 **Secret**의 키를 가리킵니다 — 스크레이프 설정에 자격증명이 섞이기 쉬워 평문 필드를 피한 설계입니다.

안 굶힐 때는 증상으로 화면을 고릅니다

여기까지가 대상을 고르는 과정입니다. 문제는 대개 두 화면 중 하나에서 갑니다.

`/service-discovery` 는 발견과 선별을, `/targets` 는 연결을 보여 줍니다. 앞 화면이 정상인데 뒤가 DOWN이면 relabeling은 제 일을 다 한 것이므로 고칠 곳은 대상 쪽입니다.

증상	볼 화면	lastError	볼 곳
타깃이 아예 없음	<code>/service-discovery</code>	connection refused	대상의 바인딩 주소
라벨이 예상과 다름	<code>/service-discovery</code>	404 Not Found	<code>metrics_path</code>
타깃은 있는데 DOWN	<code>/targets</code>	context deadline exceeded	<code>scrape_timeout</code>

`up=0` 은 실패했다는 사실만 말합니다. kubeadm 클러스터의 `connection refused` 는 대개 메트릭이 루프백에만 바인딩된 것이지 설정 실수가 아닙니다.

Exporter는 다른 시스템의 값을 번역합니다

Prometheus 형식으로 말하지 않는 대상(`/proc`, DB, 장비)을 대신 통역합니다.

메트릭 정의

이름·HELP·라벨 형상을 선언합니다.

collector

실제 값을 읽는 코드. 스크레이프마다 실행됩니다.

레지스트리

collector를 모아 `/metrics` 로 노출합니다.

`Describe` 는 등록 때 형상을, `Collect` 는 스크레이프 때 최신 값을 냅니다. 값을 미리 담아 두지 않는다는 뜻입니다.

collector는 병렬로 실행됩니다

cpu

/proc/stat

meminfo

/proc/meminfo

filesystem

마운트.파일시스템

전체 응답 시간 $\approx$ 가장 느린 collector 하나

collector 수의 **합이 아니라 최댓값**이 병목입니다. 기본 활성 collector는 45개이고, 각각 고루틴으로 동시에 돕니다.

스크레이프가 느릴 때 collector 시간을 모두 더해야 할까요?

정답: 아닙니다. 병렬이므로 가장 오래 걸린 하나를 봅니다.

메모리는 free와 available을 구분합니다

메트릭	뜻
<code>node_memory_MemTotal_bytes</code>	설치된 총 메모리
<code>node_memory_MemAvailable_bytes</code>	쓸 수 있는 메모리 (free + 회수 가능분)
<code>node_memory_MemFree_bytes</code>	즉시 쓸 수 있는 free만

free는 아무도 안 쓰는 메모리, **available**은 지금 버퍼·캐시로 쓰이지만 필요하면 회수되는 것까지 포함합니다.

대시보드나 임계값에 **MemFree** 를 쓰면 **멀쩡한 서버가 메모리 부족으로 보입니다**. 리눅스는 남는 메모리를 캐시로 쓰기 때문입니다.

loadavg 대신 pressure를 보십시오

loadavg — 쓰지 마십시오

커널 소스의 주석이 이렇게 적혀 있습니다.

"Its a silly number but people think its important."

부하 평균이 높다고 체감되는 문제가 있는 것은 아닙니다.

pressure (PSI)

커널 4.20+. 디스크 I/O·메모리·CPU 경합으로 일을 못 한 시간을 직접 잽니다.

범주

뜻

waiting

기다리는 프로세스는 있지만 스케줄러가 다른 일을 찾음

stalled

아무 일도 못 찾아 CPU 사이클을 통째로 버림 — 심각

디스크·메모리 pressure는 감시할 값으로 우선 고르십시오. CPU pressure는 stalled가 없고 기존 사용률 지표와 겹쳐 효용이 덜합니다.

타임아웃은 **duration**으로 범인을 찾습니다

`node_scrape_collector_duration_seconds`

먼저 볼 것

collector별 소요 시간. `scrape_timeout` 에 가까운 하나를 찾습니다.

오해하기 쉬운 것

`node_scrape_collector_success=0` 은 하드웨어 부재일 수도 있습니다. 실패가 아닌 경우가 많습니다.

느린 collector는 `--no-collector.<이름>` 으로 끕니다. 전부 켜 두는 것이 항상 옳지는 않습니다.

textfile collector는 짧은 작업의 기록을 노출합니다

cron 작업 → `.prom` 파일 → Node Exporter `/metrics`

왜 필요한가

백업·배치처럼 금방 끝나는 작업은 Prometheus가 긁으러 왔을 때 이미 죽어 있습니다.

필수 설정

`--collector.textfile.directory` 와 `.prom` 확장자

조용한 실패 방지 — `node_textfile_mtime_seconds` 로 파일이 오래됐는지 감시합니다. 파일이 갱신되지 않아도 옛 값은 계속 노출되기 때문입니다.

실습 관찰 · 잘못된 파일은 `node_textfile_scrape_error=1` 로 드러났지만 다른 node 메트릭은 계속 수집됐습니다.

복습: exporter를 의심하는 순서

증상	먼저 볼 것
스크래이프가 느림 / timeout	<code>collector_duration_seconds</code> 최댓값
특정 메트릭이 없음	그 collector가 켜져 있나, <code>success</code>
배치 작업 기록이 안 보임	<code>textfile</code> 디렉토리와 <code>mtime</code>
메모리가 부족해 보임	<code>MemFree</code> 를 보고 있진 않은가

전체 스크래이프 시간이 3초입니다. collector가 45개일 때 평균 소요는?

정답: 평균을 구할 수 없습니다. 병렬이라 3초는 가장 느린 하나의 시간입니다.

STEP 2 / 3

저장 — 그 값이 어떤 모양으로 쌓이는가

시계열로 저장한다 → 질의한다

메트릭·라벨·샘플

WAL · head · block

인덱스 · 컴팩션

PromQL

라벨 전체가 시계열 하나를 정합니다

메트릭

무엇을 재는가

`requests_total`

시계열

이름 + 라벨 전체

`{job="api",method="GET"}`

샘플

특정 시각의 값

`12:00 → 842`

`http_requests_total{method="GET",status="200"}`

좋은 라벨

값 종류가 제한적이고 실제로 질문에 씁니다. `method`, `status`, `job`

위험한 라벨

값이 계속 새로 생깁니다.

`user_id`, `request_id`, `trace_id`

라벨 값 하나가 달라지면 별도의 시계열이 생깁니다. 사용자 100만 명을 `user_id` 로 넣으면 시계열 100만 개 — 이것이 카디널리티 비용입니다.

타입은 관측 하나가 시계열 몇 개가 되는지를 정합니다

타입	관측 1개 → 시계열	값의 움직임	쓰는 곳
Counter	1개	오르기만	요청 수, 오류 개수
Gauge	1개	오르내림	커넥션 수, 메모리
Histogram	버킷 수 + 2	버킷마다 카운터	지연 분포, SLO
Summary	분위수 개수	관측마다 재계산	단일 인스턴스 지연

타입은 저장 방식을 바꾸지 않지만 어떤 PromQL 함수를 쓸 수 있는지를 정합니다. 뒤 두 타입은 관측 하나가 여러 시계열을 동시에 건드립니다 — 카디널리티가 여기서 다시 나옵니다.

여러 인스턴스의 p95를 합쳐 보려면 Histogram과 Summary 중 무엇일까요?

정답: Histogram입니다. Summary는 클라이언트가 미리 계산해 버려 합산할 수 없습니다.

샘플 하나는 WAL과 head로 동시에 갑니다



▶ 여기서 시연 · 02-01. 스크레이프와 저장 흐름.html 을 열어 단계를 넘겨 가며 봅니다.

WAL은 아직 블록이 아닌 데이터를 지킵니다

정상 실행

샘플이 WAL에 **먼저** 기록되고, head(메모리)에도 담깁니다. 2시간마다 head가 잘려 디스크 블록이 됩니다.

재시작

메모리의 head는 사라집니다. WAL을 **재생(replay)**해 head를 되살립니다.

WAL이 없으면 **최대 2시간치** 샘플이 재시작마다 **증발합니다**. 내구성이 여기서 옵니다.

head가 메모리에 있는데도 재시작 후 데이터가 남는 이유는?

정답: 같은 샘플이 WAL에도 디스크에 기록돼 있어 재생으로 복구되기 때문입니다.

블록은 불변이고, 그 제약이 나머지를 규정합니다

chunks

압축된 실제 샘플

index

라벨에서 시계열 위치를 찾는 역색인

meta.json

시간 범위와 통계

tombstones

삭제할 구간 표시

하고 싶은 일

불변이라 실제로 하는 일

샘플 삭제

지우지 않고 **tombstone**으로 표시

작은 블록 정리

고치지 않고 **합쳐서 새로 씀** (컴팩션)

오래된 데이터 만료

샘플이 아니라 **폴더째 삭제** (보존)

인덱스는 라벨에서 샘플까지 두 단을 거칩니다

포스팅 — 라벨 → ID

라벨 조합마다 시리즈 ID 집합. 셀렉터가 여럿이면 **교집합**입니다.

시리즈 — ID → 위치

그 ID로 chunk가 어디 있는지 찾습니다.

```
{job="kubelet"}      → [10, 11, 17]
{env="production"} → [7, 9, 11, 13, 17]
교집합              → [11, 17]    ← 실제로 읽을 대상
```

인덱스가 없으면 블록은 **읽을 수 없습니다**. 질의 속도는 이 두 단이 만듭니다.

컴팩션이 키우고 보존이 지웁니다

2시간 ×3 → 6시간 ×3 → 18시간 ×3 → 54시간

컴팩션 — 합쳐서 키움

2시간 블록마다 비슷한 인덱스가 반복되는 낭비를 덜어냅니다. 작은 블록 3개 이상이 모이면 한 단계 올라갑니다. 질의가 열 파일 수가 줄고, tombstone 구간은 이때 빠집니다.

보존 — 지움

시간·크기 임계를 넘긴 블록을 디렉토리째 삭제합니다. 블록 밖에서 안을 가리키는 참조가 없어 정리가 단순합니다.

둘 다 1분마다 돕니다. 보존이 "샘플 단위"가 아닌 이유는 앞 장의 불변성 때문입니다.

PromQL은 순간과 범위에서 출발합니다

순간 벡터 `up`

시계열마다 지금 값 하나

범위 벡터 `up[5m]`

지난 5분의 값들. 대괄호가 경계입니다.

```
rate(http_requests_total[5m])  지난 5분의 Counter → 초당 평균 증가량  
sum by (job) (rate(...))      job만 남기고 나머지 라벨을 접음
```

`by` — 닫힌 목록

남길 라벨을 적습니다. 새 라벨이 생겨도 결과가 그대로라 **변화에 강합니다**.

`without` — 열린 목록

버릴 라벨을 적습니다. 새 라벨이 생기면 **결과가 쪼개집니다**.

그래프의 구멍은 대개 staleness입니다

질의는 5분(`lookback delta`)을 거슬러 값을 찾고, 없으면 시계열이 사라진 것처럼 보입니다.

화면	실제 상황	해야 할 일
선 없음	그 사유의 실패가 0건	아무것도 안 해도 됨
선 없음	앱이 죽어 메트릭을 못 냄	지금 대응

같은 화면인데 해야 할 일이 정반대입니다. 값이 0이어도 같은 메트릭을 계속 내보내야 둘이 같립니다 — 0인 평평한 선 자체가 "살아 있다"는 신호입니다.

실무 수칙 · 조건이 맞을 때만 메트릭을 등록하는 코드는 그래프에 구멍을 냅니다. 앱 시작 시 미리 등록하십시오.

배운 PromQL로 실제 값을 읽어 봅니다

Node Exporter의 CPU는 백분율이 아니라 **각 모드에서 보낸 초**로 나옵니다.

```
sum without (cpu, mode) (  
  rate(node_cpu_seconds_total{mode!="idle"}[5m])  
)
```

결과의 뜻

비유휴 시간의 비율. CPU가 하나면 최대 **1.0**, 둘이면 **2.0** — 코어 수만큼 커 집니다.

0~100%로 정규화

코어 수로 나눕니다.

```
count without (cpu, mode)  
(node_cpu_seconds_total{mode="idle"})
```

mode="idle" 은 취향이 아니라 **필수**입니다. 이 메트릭은 (코어 × 모드) 만큼의 시계열이라, 모드를 하나로 고정해야 **count** 가 코어 수가 됩니다.

STEP 3 / 3

알림 — 조건이 참일 때 누구에게 어떻게 닿는가

규칙을 평가한다 → 묶어서 보낸다

inactive · pending · firing

그룹핑 · 라우팅

억제 · 시간 구간

룰 테스트 · HA

알림은 두 프로그램이 나눠 말합니다

Prometheus

- PromQL 규칙 평가
- inactive / pending / firing
- firing 알림을 전달

Alertmanager

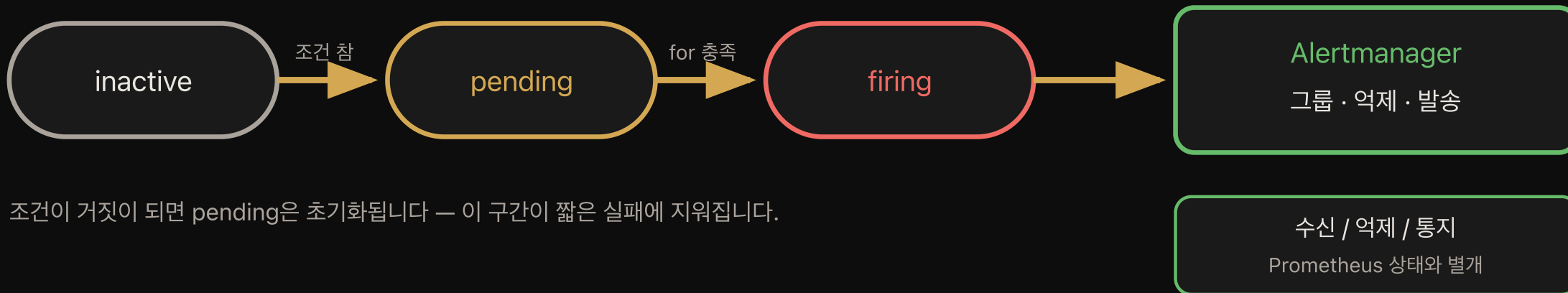
- 그룹핑과 라우팅
- 억제와 침묵
- Slack·메일 발송

이 장의 단 하나의 메시지 — 두 상태는 다른 것입니다. Prometheus는 알림을 보내지 않습니다.

실습 관찰 · Alertmanager를 멈춰도 Prometheus의 firing은 유지됐고 새 통지만 끊겼습니다.

for 는 순간 이상과 지속 이상을 가릅니다

PROMETHEUS · 규칙 평가



조건이 거짓이 되면 pending은 초기화됩니다 — 이 구간이 짧은 실패에 지워집니다.

Kind 실습 · 평가 주기 30초보다 짧은 **for: 10s** 는 다음 평가에서 바로 firing됐고, **for: 2m** 은 pending이 보였습니다.

▶ 여기서 시연 · 02-02.알림 흐름.html 에서 세 시나리오를 비교합니다.

for 를 얼마로 둘지에는 경험칙이 있습니다

warning

15분보다 짧지 않게. 1~2시간짜리 for 를 겁내지 마십시오.

critical

15분보다 길지 않게. 짧은 for (0m ~ 5m)는 여기에 아껴 둡니다.

놓치기 쉬운 제약 — for 는 룰 평가 주기보다 길어야 의미가 있습니다. 5분마다 평가하면서 for: 3m 을 걸면 그 3분은 아무 일도 하지 않습니다.

반대편 손잡이 keep_firing_for

for 가 발화 전 대기라면, 이것은 조건이 거짓이 된 뒤에도 firing을 유지하는 시간입니다.

왜 필요한가

조건이 참·거짓을 오갈 때 해소와 발화가 번갈아 쏟아지는 것을 막습니다.

짧은 실패가 `for` 를 되돌릴 수 있습니다

조건 14분 참 → 스크레이프 한 번 실패 → pending 0분

왜 생기나

데이터 공백이 조건을 거짓처럼 만듭니다. 앞 장의 staleness가 알림에서 되돌아오는 자리입니다.

보강

`last_over_time()` 으로 짧은 공백을 건넵니다. `max_over_time()` 은 추가 지연이 붙습니다.

허용 구간은 진짜 장애를 숨기지 않도록 의도적으로 정합니다. 무작정 늘리면 탐지가 늦어집니다.

그룹핑은 폭풍을 한 묶음으로 바꿉니다

노드 100대가 한꺼번에 죽으면 메일 100통이 아니라 **한 통**이어야 합니다.

group_wait

첫 알림 뒤 같은 그룹을 모으는 시간
기본 30초

group_interval

같은 그룹에 **변경**이 생겼을 때 다음 통지까지
기본 5분

repeat_interval

변화가 없어도 **다시** 알리는 주기
기본 4시간

실습 관찰 · `group_by: [alertname]` 에서 같은 값의 firing 두 건이 `group_wait` 안에 들어와 메일 한 통 `[FIRING:2]` 이 되었습니다.

라우트 트리와 시간 구간이 수신자를 정합니다

```
severity="critical" → on-call  
team="payments"    → payments-slack
```

기본 동작

첫 번째로 맞는 자식 라우트에서 멈춥니다.

`continue: true`

맞은 뒤에도 다음 형제 라우트를 계속 확인합니다.

`mute_time_intervals`

이 시간대에는 발송하지 않습니다. warning을 업무 시간에만 받을 때.

`active_time_intervals`

이 시간대에만 발송합니다.

시간 구간은 최상위 라우트에는 걸 수 없습니다. 구간 밖이면 발송만 멈출 뿐 라우팅 탐색은 계속됩니다.

억제는 원인이 결과 알림을 가립니다

DB가 죽으면 API 오류도 함께 터집니다. 당직자가 볼 것은 **원인** 하나입니다.

source

가리는 쪽

DBDown

target

가려지는 쪽

APIHighError

equal

값이 같아야 할 라벨

cluster, service

`equal` 이 없으면 다른 클러스터의 장애까지 가려집니다. 하위 알림은 Prometheus에서 여전히 `firing`이고 통지만 막힙니다.

`promtool test rules`로 알리를 테스트합니다

```
promtool check rules rule.yaml    문법만 검사
promtool test rules unit.yaml     기대대로 발화하는지 검사
```

왜 `amtool` 이 아닌가

알림은 `Prometheus` 안에서 정의되고 평가되기 때문입니다. `amtool`은 Alertmanager 쪽 검증입니다.

무엇을 단언하나

시계열을 지어 넣고 "몇 분에 무엇이 어떤 라벨로 발화해야 하는지"를 적습니다.

사고 시간대에 쿼리를 돌려 보는 방식은 유효하지만, **룰을 손본 뒤에도 계속 동작하리라는 보장은 주지 못합니다.** 알림도 코드입니다.

Alertmanager 앞에 로드밸런서를 두지 마십시오

가십이 공유하는 것

침묵, 그리고 어떤 그룹을 이미 보냈는지. 덕분에 같은 알림이 여러 번 가지 않습니다.

공유하지 않는 것

어떤 알림을 받았는지.

그래서 Prometheus가 모든 인스턴스에 직접 보내야 합니다.

LB로 한 대만 받게 하면, 그 인스턴스가 죽었을 때 나머지는 알림의 존재 자체를 몰라 대신 보낼 수 없습니다.

합의 알고리즘을 쓰지 않아 정족수가 필요 없습니다. 보통 2~3대를 서로 다른 AZ에 둡니다.

한 번의 스크레이프가 끝까지 가는 길

1. 서비스 디스커버리가 후보를 냅니다.
2. `relabel_configs` 가 굵을 대상을 고릅니다.
3. Exporter의 collector가 값을 만듭니다.
4. `metric_relabel_configs` 가 저장할 시계열을 고릅니다.
5. TSDB가 **WAL** · **head** · **block**에 저장합니다.
6. 규칙 평가 뒤 firing 알림이 **Alertmanager**로 갑니다.

오늘 본 세 구간이 각각 이 흐름의 한 토막이었습니다 — 수집(1~4) · 저장(5) · 알림(6).

장애가 보이지 않을 때 확인할 경계

증상	경계	도구 / 신호
타킷 없음	SD → target relabel	<code>/service-discovery</code> , <code>/targets</code>
scrape timeout	collector → exporter	<code>collector_duration</code>
시계열 없음	metric relabel → TSDB	PromQL, 설정
그래프에 구멍	staleness	0을 계속 내보내는가
알림 안 옴	Prometheus → Alertmanager	pending/firing, route, inhibit

어느 경계에서 사라졌는지에 따라 원인도 고칠 곳도 달라집니다.

마지막 5문장 회상

1. 라벨 전체가 시계열 하나를 정합니다.
2. 샘플은 **WAL** → **head** → **block**으로 이동하고, 블록은 불변입니다.
3. relabeling은 타깃과 시계열을 서로 다른 위치에서 거릅니다.
4. Prometheus의 **firing**과 Alertmanager의 **억제**는 다른 상태입니다.
5. Node Exporter timeout은 가장 느린 **collector**의 duration부터 봅니다.

보지 않고 설명할 수 있다면 네 장의 뼈대를 잡은 것입니다.

다음은 한 대를 넘어서는 이야기입니다 — 06장 샤딩·HA · 07장 카디널리티 디버깅 · 09장 remote write · 10장 Thanos.